



Ssi User's Manual for FC7300F8MDQ

Rev: 1.2.0

Target Devices:
FC7300F8MDQ

Table of Contents

Chapter 1. Ssi Introduction	3
1.1. Requirements	3
1.2. Introduction to CDD	3
1.3. Hardware Summary	3
Chapter 2. Software Design	5
2.1. Deviation form Requirements	5
2.2. File Structure	5
2.3. Define and Macro reference	5
2.3.1. Macros in CDD_Ssi.c	5
2.3.2. Macros in CDD_Ssi.h	5
2.4. Enum reference	6
2.4.1. Ssi_StatusType	6
2.4.2. Ssi_ReturnType	7
2.4.3. Ssi_ProtocolModeType	7
2.4.4. Ssi_ChnnlSelType	7
2.4.5. Ssi_ResSelType	8
2.4.6. Ssi_ChnnlSwitchType	8
2.5. Structures and typedefs reference	8
2.5.1. Ssi_MessageDataType	8
2.5.2. Ssi_NotifactionType	9
2.5.3. Ssi_SubInstanceConfigType	9
2.5.4. Ssi_InstanceConfigType	10
2.5.5. Ssi_ConfigType	11
2.6. Function reference	11
2.6.1. Ssi_Init	11
2.6.2. Ssi_DeInit	11
2.6.3. Ssi_GetVersionInfo	12
2.6.4. Ssi_MainFunctionMessageRead	12
2.7. API Sequence Diagram	12
2.7.1. Initialization	12
2.7.2. De-Initialization	13
2.7.3. Interrupt mode	13
2.7.4. Polling mode	13
Chapter 3. Tresos Configuration Items	15
3.1. Container Inclusion Relation	15
3.2. Containers and Variables	15
3.2.1. IMPLEMENTATION_CONFIG_VARIANT	15
3.2.2. SsiGeneral	15
3.2.2.1. SsiDevErrorDetect	16
3.2.2.2. SsiMulticoreSupport	16
3.2.2.3. SsiDeInitApi	16
3.2.2.4. SsiVersionInfoApi	16
3.2.2.5. SsiIndex	17
3.2.3. SsiEcucPartitionRef	17
3.2.4. SsiConfigSet	17

3.2.4.1.	SsiControllerConfig	18
3.2.4.2.	SsiControllerId	18
3.2.4.3.	SsiHWInstance	18
3.2.4.4.	SsiControllerEcucPartitionRef	18
3.2.4.5.	SsiModuleClockRef	19
3.2.4.6.	SsiModuleClock	19
3.2.4.7.	SsiPrescaler	19
3.2.4.8.	SsiLowGlobalCounterWindows	19
3.2.4.9.	SsiHighGlobalCounterWindows	20
3.2.4.10.	SsiControllerActivation	20
3.2.4.11.	SsiSubinstanceConfig	20
3.2.4.12.	SsiSubinstanceId	20
3.2.4.13.	SsiSubinstanceActivation	21
3.2.4.14.	SsiHWSubinstance	21
3.2.4.15.	FilterEnable	21
3.2.4.16.	FilterWidth	21
3.2.4.17.	ProtocolSelect	22
3.2.4.18.	ChannelSelect	22
3.2.4.19.	ResistanceSelect	22
3.2.4.20.	ChannelSwitch	22
3.2.4.21.	WindowsRangeCheckEnable	23
3.2.4.22.	GeneralPWMTIMEOUTCounter	23
3.2.4.23.	GeneralPWMPolarity	23
3.2.4.24.	MsgReadType	23
3.2.4.25.	SsiNotification	24
3.2.4.26.	QDTCControllerId	24

Chapter 1. Ssi Introduction

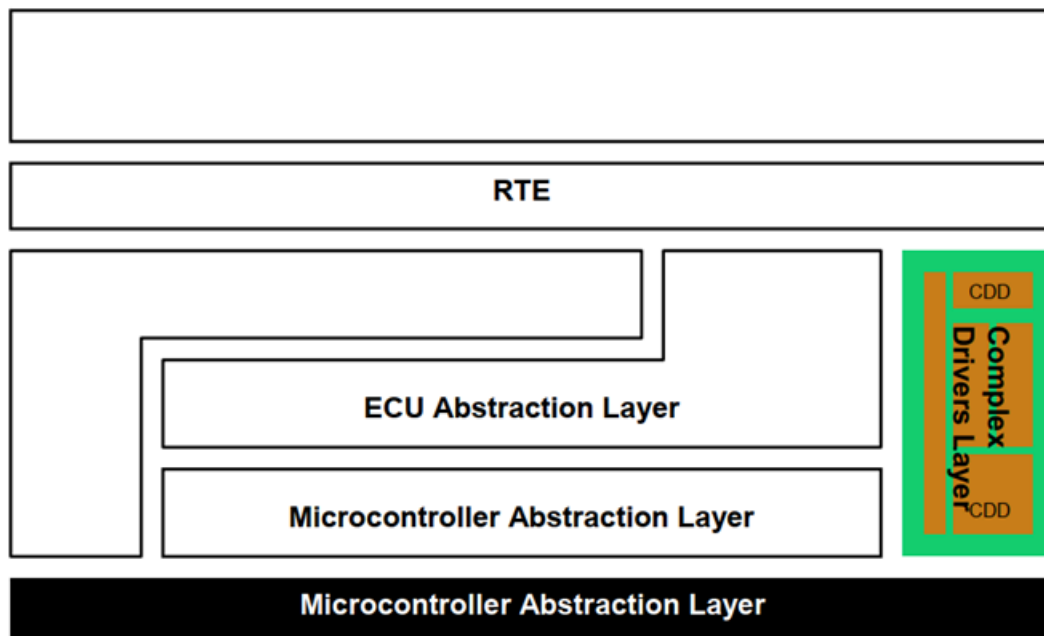
1.1. Requirements

The design of this module follows the specifications of the complex driver specified in AUTOSAR Classic Platform Release R20-11. For detailed requirements, refer to the AUTOSAR_EXP_CDDDesignAndIntegrationGuideline.pdf.

1.2. Introduction to CDD

A Complex Driver is a software entity not standardized by AUTOSAR that can access or be accessed via AUTOSAR Interfaces and/or Basic Software Modules APIs. According to the autosar Layered Software Architecture, a CDD is a specific module located in the Complex Drivers Layer of the Basic Software which interacts with standard BSW modules or Rte.

- A CDD may need to interface to modules of the layered software architecture
- A module of the layered software architecture may need to interface to a CDD
- A CDD may need to interface SW-Cs via RTE



1.3. Hardware Summary

This SSI is a speed and sensor interface that supports 4 sub-instances. It can decode multiple wheel speed sensors (Standard, PWM, and AK), as well as general PWM cycle and pulse measurements. The results can be read from registers. The SSI also supports multiple input sources and configurable interrupts. If the system recovers from stop mode or SSI is enabled, the first transmission valid flag is ignored.

- Supports wheel speed sensor measurements.
 1. Each sub-instance has an independent internal comparator and supports analog signal inputs from wheel speed sensors. The comparator has two outputs: *inm* and *inh*, and is able to differentiate between currents of 7mA (*inm*=*inh*=0), 14mA (*inm*=1, *inh*=0) and 28mA (*inm*=1, *inh*=1).
 2. Standard, PWM and AK wheel speed protocols are supported.
 3. PWM and AK protocols can directly query the decoded value.

4. External resistors are selectable and support 30, 50 ohms for AK protocol and 30, 50, 60 and 100 ohms for PWM and Standard protocols.
 5. The output of the internal comparator can be observed.
 6. Internal CMP supports BIST mode, which can be self-tested by configuring the reference voltage.
- Supports general PWM signal measurement.
 1. Supports polarity selection of input PWM.
 2. Supports input from pad or TRGSEL.
 3. Supports up to 24-bit cycle count and 18-bit pulse count.
 4. Configurable timeout setting.
 - Interrupt Generation
 1. Previous protocol transfer completed.
 2. Protocol counter timeout. s
 3. Protocol counter exceeds the user allowable range, and flag set at the end of the current transmission.
 4. Wheel Protocol stopped, and flag set at the end of the current transmission.
 5. Wheel AK/PWM protocol has a decoding error, and flag set at the end of the current transmission.
 6. Wheel AK/PWM Protocol has an interval error, and flag set at the end of the current transmission.
 7. Wheel AK/PWM protocol has a pulse width error, and flag set at the end of the current transmission.
 - General Configuration
 1. Input filters are supported.

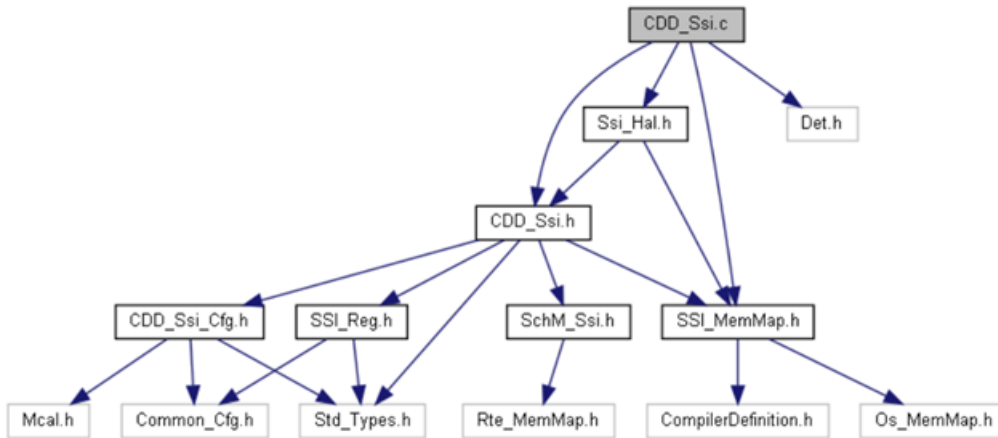
FC7300F8MDQ series have 6 ADC instances and each instance support fixed number of logical ADC channels. External physical channels and internal physical channels can be assigned to these logical ADC channels.

Chapter 2. Software Design

2.1. Deviation form Requirements

N/A.

2.2. File Structure



2.3. Define and Macro reference

2.3.1. Macros in CDD_Ssi.c

- #define SSI_VENDOR_ID_C 174
- #define SSI_AR_RELEASE_MAJOR_VERSION_C 4
- #define SSI_AR_RELEASE_MINOR_VERSION_C 6
- #define SSI_AR_RELEASE_REVISION_VERSION_C 0
- #define SSI_SW_MAJOR_VERSION_C 1
- #define SSI_SW_MAJOR_VERSION_C 2
- #define SSI_SW_MAJOR_VERSION_C 0

2.3.2. Macros in CDD_Ssi.h

- #define SSI_VENDOR_ID 174
- #define SSI_MODULE_ID 255
- #define SSI_AR_RELEASE_MAJOR_VERSION 4
- #define SSI_AR_RELEASE_MINOR_VERSION 6
- #define SSI_AR_RELEASE_REVISION_VERSION 0
- #define SSI_SW_MAJOR_VERSION 1
- #define SSI_SW_MINOR_VERSION 2

- #define SSI_SW_PATCH_VERSION 0

- #define SSI_E_ALREADY_UNINITIALIZED_U8 ((uint8)0x0A)

API SSI_Ssi_DeInit service called when the SSI driver and the Hardware are already uninitialized

- #define SSI_E_UNINIT_U8 ((uint8)0x0B)

API service used without module initialization

- #define SSI_E_PARAM_U8 ((uint8)0x0C)

API service called with wrong parameter.

- #define SSI_E_ALREADY_INITIALIZED_U8 ((uint8)0x0D)

API SSI_Ssi_Init service called when the SSI driver and the Hardware are already initialized.

- #define SSI_E_INIT_FAILED_U8 ((uint8)0x0EU)

API SSI_Init service execution failed.

- #define SSI_E_PARTITION_MAPPING ((uint8)0x0FU)

API service called with wrong core ID.

- #define SSI_E_TIMEOUT_U8 ((uint8)0x10U)

API service called with wrong core ID.

- #define SSI_INIT_ID_U8 ((uint8)0x1U)

API service ID for Ssi_Init function.

- #define SSI_DEINIT_ID_U8 ((uint8)0x2U)

API service ID for Ssi_DeInit function.

- #define SSI_GETVERSIONINFO_ID_U8 ((uint8)0x3U)

API service ID for Ssi_GetVersionInfo function.

- #define SSI_MSG_READ_ID_U8 ((uint8)0x4U)

API service ID for Ssi_RequestSpcPulse function.

2.4. Enum reference

2.4.1. Ssi_StatusType

Defines the SSI Status type.

```
typedef enum
{
    SSI_UNINIT = 0U,
```

```
SSI_INITIALIZED  
} Ssi_StatusType;
```

Value Meaning

Value	Description
SSI_UNINIT	The SSI is not initialized
SSI_INITIALIZED	The SSI has been initialized

2.4.2. Ssi_ReturnType

Defines the SSI Function Call Result.

```
typedef enum  
{  
    SSI_OK = 0U,  
    SSI_NOT_OK  
} Ssi_ReturnType;
```

Value Meaning

Value	Description
SSI_OK	Call Correctly
SSI_NOT_OK	Call Error return

2.4.3. Ssi_ProtocolModeType

Defines the different protocol modes supported by the SSI driver.

```
typedef enum  
{  
    SSI_NORMAL = 0x0U,  
    SSI_PWM = 0x1U,  
    SSI_AK = 0x2U,  
    SSI_GPWM = 0x3U  
} Ssi_ProtocolModeType;
```

Value Meaning

Value	Description
SSI_NORMAL	Normal protocol mode
SSI_PWM	PWM protocol mode
SSI_AK	AK protocol mode
SSI_GPWM	GPWM protocol mode

2.4.4. Ssi_ChnnlSelType

Defines the SSI channel selection type.

```
typedef enum  
{  
    SSI_INTERNAL_CMP = 0x0U,
```



```
SSI_DIGITAL_PAD = 0x1U,  
SSI_ONCHIP_CMP = 0x2U,  
SSI_TRGSEL = 0x3U  
} Ssi_ChnnlSelType;
```

Value Meaning

Value	Description
SSI_INTERNAL_CMP	Internal comparator mode
SSI_DIGITAL_PAD	Digital pad mode
SSI_ONCHIP_CMP	On-chip comparator mode
SSI_TRGSEL	Trigger select mode

2.4.5. Ssi_ResSelType

Defines the SSI resistance selection type.

```
typedef enum  
{  
    SSI_RES_50OHM = 0x0U,  
    SSI_RES_30OHM = 0x1U  
} Ssi_ResSelType;
```

Value Meaning

Value	Description
SSI_RES_50OHM	50 Ohm resistance mode
SSI_RES_30OHM	30 Ohm resistance mode

2.4.6. Ssi_ChnnlSwitchType

Defines the SSI channel switch type.

```
typedef enum  
{  
    SSI_INM_VLD = 0x0U,  
    SSI_INH_VLD = 0x1U  
} Ssi_ChnnlSwitchType;
```

Value Meaning

Value	Description
SSI_INM_VLD *	INM mode valid
SSI_INH_VLD **	INH mode valid

2.5. Structures and typedefs reference

2.5.1. Ssi_MessageDataType

Structure for message buffer..

```
typedef struct
{
    uint8 u8Subidx;
    uint8 u8Status;
    uint8 u8Prot;
    uint32 U32Whlval;
    uint32 u32Date1;
    uint32 u32Date2;
} Ssi_MessageDataType;
```

Data Fields

Type	Member	Description
uint8	u8Subidx	Sub-index of the message
uint8	u8Status	Status of the message
uint8	u8Prot	Protocol type of the message
uint32	U32Whlval	Message Wheel value
uint32	u32Date1	Message data1 value
uint32	u32Date2	Message data2 value

2.5.2. Ssi_NotifactionType

This type shall be chosen in order to implement slow message received callback function.

```
typedef void (*Ssi_NotifactionType)(Ssi_MessageDataType *pMsg);
```

2.5.3. Ssi_SubInstanceConfigType

Structure to configure the SSI sub-instance.

```
typedef struct
{
    uint8      u8SubInstanceLogicId;
    uint8      u8SubInstanceHwId;
    uint8      u8GpwmToc;
    boolean    bSubInstanceEn;
    boolean    bInterruptEn;
    boolean    bFilterEn;
    boolean    bIntCmpEn;
    boolean    bWindowsRangeCheck;
    boolean    bGpwmInv;
    Ssi_ProtocolModeType eProtocolMode;
    Ssi_ChnnlSelType   eChnlSelect;
    Ssi_ResSelType     eResSelect;
    Ssi_ChnnlSwitchType eChnlSwitch;
    Ssi_NotifactionType pNotification;
    uint32      u32FiltWidth;
} Ssi_SubinstanceConfigType;
```

Data Fields

Type	Member	Description
uint8	u8SubInstanceLogicId	Logical ID of the sub-instance

Type	Member	Description
uint8	u8SubInstanceHwId	Hardware ID of the sub-instance
uint8	u8GpwmToc	The general PWM timeout counter
boolean	bSubInstanceEn	Sub-instance enable flag
boolean	bInterruptEn	Interrupt enable flag
boolean	bFilterEn	Filter enable flag
boolean	bIntCmpEn	SIInternal comparison enable flag
boolean	bWindowsRangeCheck	Windows range check enable flag
boolean	bGpwmInv	Invert the general PWM input enable flag
Ssi_ProtocolModeType	eProtocolMode	Protocol mode selection
Ssi_ChnnlSelType	eChnlSelect	Channel selection mode
Ssi_ResSelType	eResSelect	Resistance selection mode
Ssi_ChnnlSwitchType	eChnlSwitch	Channel switch mode
Ssi_NotifactionType	pNotification	Notification function pointer
uint32	u32FiltWidth	Filter width

2.5.4. Ssi_InstanceConfigType

Structure to configure the SSI instance.

```
typedef struct
{
    uint8          u8InstanceLogicIndex;
    uint8          u8InstanceHwIndex;
    uint8          u8SubInstanceCount;
    uint8          u8DivClock;
    boolean        bControllerEn;
    uint32         u32LowGlobalCounterWindows;
    uint32         u32HighGlobalCounterWindows;
    Ssi_SubinstanceConfigType **pSSISubInstanceCfg;
} Ssi_InstanceConfigType;
```

Data Fields

Type	Member	Description
uint8	u8InstanceLogicIndex	Logical index of the instance
uint8	u8InstanceHwIndex	Hardware index of the instance
uint8	u8SubInstanceCount	Number of sub-instances
uint8	u8DivClock	Divider clock setting
boolean	bControllerEn	Controller enable flag
uint32	u32LowGlobalCounterWindows	Low global window value
uint32	u32HighGlobalCounterWindows	High global window value
Ssi_SubinstanceConfigType **	pSSISubInstanceCfg	Pointer to an array of sub-instance configurations

2.5.5. Ssi_ConfigType

Data structure containing the set of configuration parameters required for initializing.

```
typedef struct
{
    uint8 u8InstanceCount;
    Ssi_InstanceConfigType **pSSIInstanceCfg;
    boolean *SSI_CoresMappingPtr;
    uint32 *SSI_CtrlCoresMappingPtr;
} Ssi_ConfigType;
```

Data Fields

Type	Member	Description
uint8	u8InstanceCount	The counter of the used SSI instance
Ssi_InstanceConfigType **	pSSIInstanceCfg	The Pointer array of the SSI_InstanceConfigType
boolean *	SSI_CoresMappingPtr	The Pointer array of the core mapping, if true, means the current core need to manage at least one ssi instance
uint32 *	SSI_CtrlCoresMappingPtr	The Pointer array of the controller mapping, the value indicate which core should manage the ssi instance

2.6. Function reference

2.6.1. Ssi_Init

Initializes the Ssi hardware unit and the driver.

```
void Ssi_Init(const Ssi_ConfigType *ConfigPtr)
```

Parameters

Type	Name	Direction	Description
Ssi_ConfigType *	Configptr	input	Pointer to configuration set in Variant PB (Variant PC requires a NULL_PTR).

Returns

void

2.6.2. Ssi_DeInit

De-initializes the Ssi hardware unit and the driver.

This service is a non reentrant function. Returns all underlying hardware to a state comparable to their power on reset state, and de-initialize the SSI driver. This service is a non-reentrant function. Returns all underlying hardware to a state comparable to their power on reset state, and de-initialize the SSI driver.

```
void Ssi_DeInit(void)
```

Parameters

void

Returns

void

2.6.3. Ssi_GetVersionInfo

Initializes the Ssi hardware unit and the driver.

```
void Ssi_GetVersionInfo(Std_VersionInfoType *pVersioninfo)
```

Parameters

Type	Name	Direction	Description
Std_VersionInfoType *	pVersioninfo	output	Pointer to location to store version info.

Returns

void

2.6.4. Ssi_MainFunctionMessageRead

This function performs the polling of reading the message when MsgReadType is set to Polling.

```
void Ssi_MainFunctionMessageRead(void)
```

Parameters

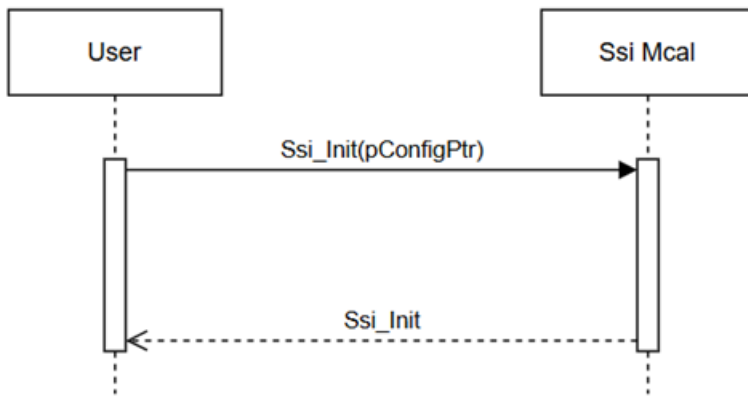
void

Returns

void

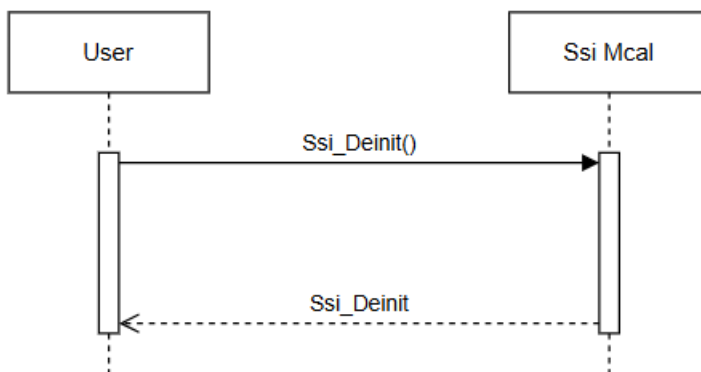
2.7. API Sequence Diagram**2.7.1. Initialization**

Ssi Driver Initialization.



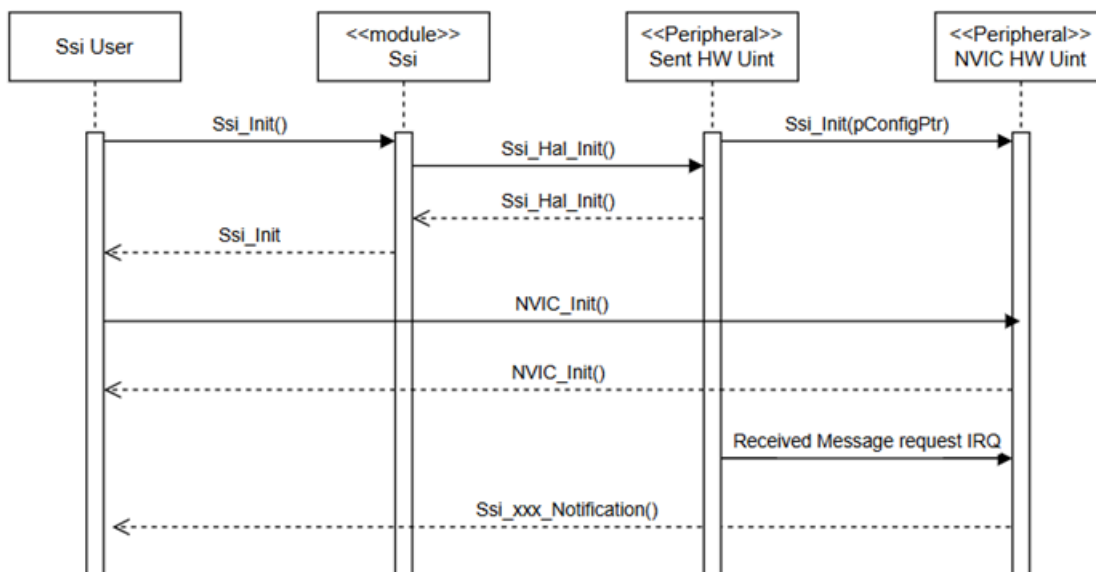
2.7.2. De-Initialization

Ssi Driver De-Initialization.



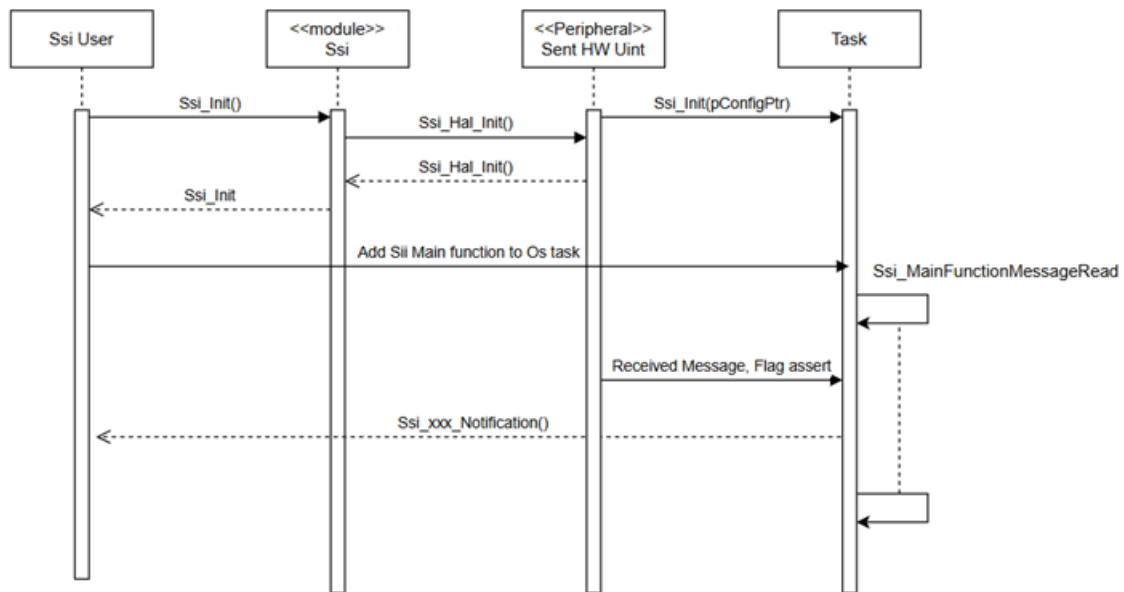
2.7.3. Interrupt mode

Ssi Driver work in interrupt mode.



2.7.4. Polling mode

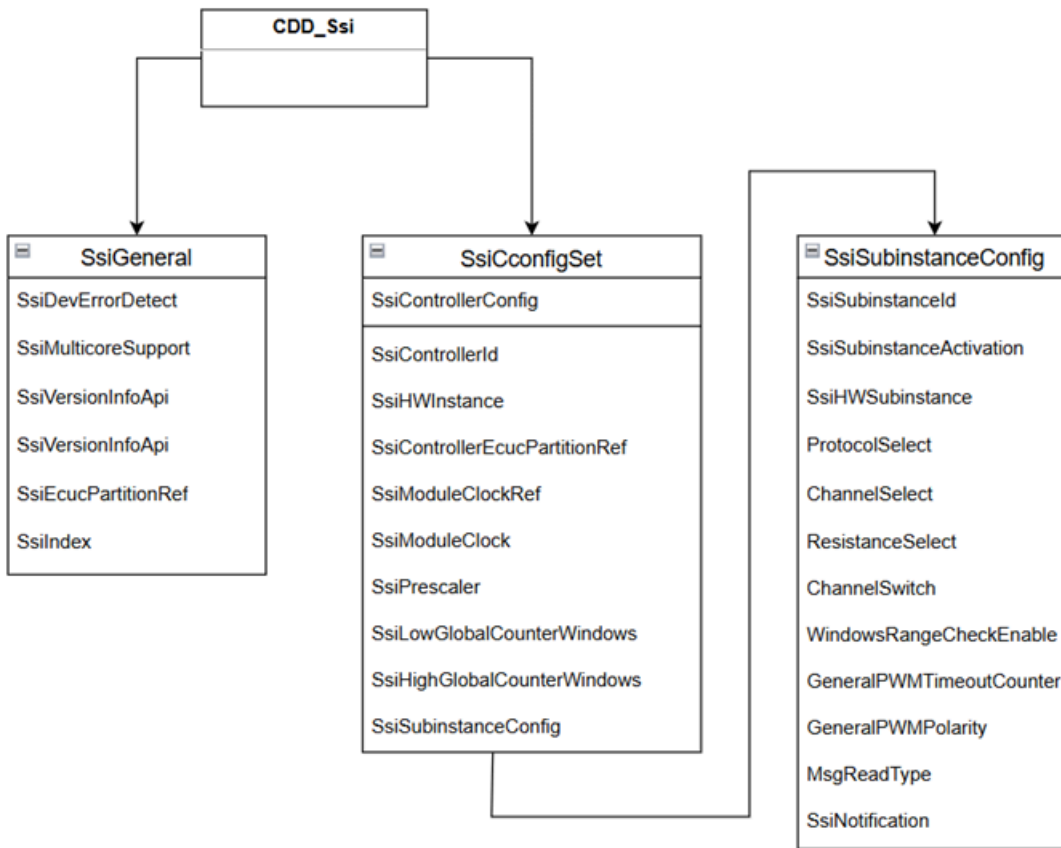
Ssi Driver work in polling mode.



Chapter 3. Tresos Configuration Items

3.1. Container Inclusion Relation

The contain inclusion relation is shown as below:



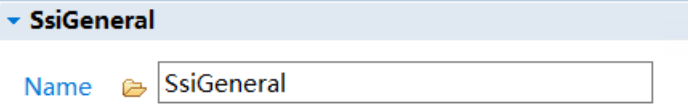
3.2. Containers and Variables

3.2.1. IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Config Variant
	Default	VariantPreCompile

3.2.2. SsiGeneral

Container	SsiGeneral
Description	Vendor specific: Configuration of general Ssi parameters.

Screenshot		
	▼ SsiGeneral Name  SsiGeneral	
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.2.1. SsiDevErrorDetect

Container	SsiDevErrorDetect	
Description	Switches the Development Error Detection and Notification on or off.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	SsiDev Error Detect
	Default	false

3.2.2.2. SsiMulticoreSupport

Container	SsiMulticoreSupport	
Description	Switches multicore support on or off.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	SsiMulticoreSupport
	Default	false

3.2.2.3. SsiDeInitApi

Container	SsiDeInitApi	
Description	Vendor specific: Enables/Disables the Ssi_DeInit API.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Provide De-Init API
	Default	false

3.2.2.4. SsiVersionInfoApi

Container	SsiVersionInfoApi
-----------	-------------------

Description	Switches the Ssi_GetVersionInfo() API: ON or OFF.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Provide Version Info API
	Default	false

3.2.2.5. SsiIndex

Container	SsiIndex	
Description	Specifies the InstanceId of this module instance.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Ssi Driver Index
	Default	0

3.2.3. SsiEcucPartitionRef

Container	SsiEcucPartitionRef	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	LIST
	Label	N/A
	Default	N/A

3.2.4. SsiConfigSet

Container	SsiConfigSet	
Description	Vendor specific: Configuration of general Ssi parameters.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.4.1. SsiControllerConfig

Container	SsiControllerConfig	
Description	The container contains the the entire ssi instance config of each Ssi controller.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Ssi Controller ID
	Default	N/A

3.2.4.2. SsiControllerId

Container	SsiControllerId	
Description	This parameter provides the controller ID which is unique in a given SSI Driver.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Ssi Controller ID
	Default	N/A

3.2.4.3. SsiHWInstance

Container	SsiHWInstance	
Description	Specifies which one of the on-chip SSI interfaces is associated with this controller ID.	
Screenshot		
Properties	Property	Value
	Type	STRING(RANGE)
	Label	Ssi hardware Instance
	Default	SSI0

3.2.4.4. SsiControllerEcucPartitionRef

Container	SsiControllerEcucPartitionRef	
Description	Maps a SSI controller to zero or multiple ECUC partitions to limit the access to this controller.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Label	SsiControllerEcucPartitionRef
	Default	N/A

3.2.4.5. SsiModuleClockRef

Container	SsiModuleClockRef	
Description	Reference to the CPU clock configuration, which is set in the MCU driver configuration.	
Screenshot	Ssi Module Reference Clock 	
Properties	Property	Value
	Type	REFERENCE
	Label	Ssi Module Clock
	Default	N/A

3.2.4.6. SsiModuleClock

Container	SsiModuleClock	
Description	Reference to the CPU clock configuration, which is set in the MCU driver configuration.	
Screenshot	Ssi Module Clock (Hz) 	
Properties	Property	Value
	Type	REFERENCE
	Label	Ssi Module Clock
	Default	N/A

3.2.4.7. SsiPrescaler

Container	SsiPrescaler	
Description	Ssi Clock Divider.	
Screenshot	Ssi Prescaler (Hz) 	
Properties	Property	Value
	Type	INTEGER
	Label	Ssi Prescaler (Hz)
	Default	6

3.2.4.8. SsiLowGlobalCounterWindows

Container	SsiLowGlobalCounterWindows	
Description	Global Counter Windows Low Value.	
Screenshot	Low Global Counter Windows (0 -> 16777215) 	
Properties	Property	Value
	Type	INTEGER
	Label	Low Global Counter Windows (0 -> 16777215)
	Default	1

3.2.4.9. SsiHighGlobalCounterWindows

Container	SsiHighGlobalCounterWindows	
Description	Global Counter Windows High Value.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	High Global Counter Windows (0 -> 16777215)
	Default	16777215

3.2.4.10. SsiControllerActivation

Container	SsiControllerActivation	
Description	Defines if a SSI controller is used in the configuration.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Ssi Controller Activation
	Default	true

3.2.4.11. SsiSubinstanceConfig

Container	SsiSubinstanceConfig	
Description	This container contains the configuration (parameters) of the SSI subinstances.	
Screenshot		
Properties	Property	Value
	Type	MAP
	Label	N/A
	Default	N/A

3.2.4.12. SsiSubinstanceId

Container	SsiSubinstanceId	
Description	This parameter provides the Subinstance ID which is unique in a given SSI subinstance.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Ssi Subinstance ID
	Default	0

3.2.4.13. SsiSubinstanceActivation

Container	SsiSubinstanceActivation	
Description	Defines if a SSI Subinstance is used in the configuration.	
Screenshot	Ssi Subinstance Activation  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Ssi Subinstance Activation
	Default	true

3.2.4.14. SsiHWSubinstance

Container	SsiHWSubinstance	
Description	Specifies which one of the on-chip SSI interfaces is associated with this controller ID.	
Screenshot	Ssi hardware Subinstance  Ssi_SubInstance1 	
Properties	Property	Value
	Type	STRING(RANGE)
	Label	Ssi hardware Subinstance
	Default	Ssi_Subinstance0

3.2.4.15. FilterEnable

Container	FilterEnable	
Description	Maps a SSI controller to zero or multiple ECUC partitions to limit the access to this controller.	
Screenshot	Enable Filter  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Filter
	Default	TRUE

3.2.4.16. FilterWidth

Container	FilterWidth	
Description	Specifies the filter width is associated with this controller ID.	
Screenshot	Filter Width (0 -> 15)  15 	
Properties	Property	Value
	Type	INTEGER
	Label	Filter width

	Default	1
--	----------------	---

3.2.4.17. ProtocolSelect

Container	ProtocolSelect	
Description	Specifies which Protocol is associated with this subinstance ID.	
Screenshot		
Properties	Property	Value
	Type	STRING(RANGE)
	Label	Protocol select
	Default	SSI_NORMAL

3.2.4.18. ChannelSelect

Container	ChannelSelect	
Description	Specifies which Channel Selected is associated with this subinstance ID.	
Screenshot		
Properties	Property	Value
	Type	STRING(RANGE)
	Label	Channel Select
	Default	SSI_INTERNAL_CMP

3.2.4.19. ResistanceSelect

Container	ResistanceSelect	
Description	Specifies which Resistance Selected is associated with this controller ID, only be used by the internal CMP.	
Screenshot		
Properties	Property	Value
	Type	STRING(RANGE)
	Label	Select Resistance
	Default	SSI_RES_50OHM

3.2.4.20. ChannelSwitch

Container	ChannelSwitch	
Description	Specifies which channel switch is associated with this controller ID, only used for non-AK protocols, and is invalid for the AK protocol. For internal CMP, double external resistance should be used.	
Screenshot		
Properties	Property	Value

	Type	STRING(RANGE)
	Label	Switch Channel
	Default	SSI_INM_VLD

3.2.4.21. WindowsRangeCheckEnable

Container	WindowsRangeCheckEnable	
Description	If enabled, the windows range check is enabled.	
Screenshot	Enable Windows Range Check 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable Windows Range Check
	Default	false

3.2.4.22. GeneralPWMTimeoutCounter

Container	GeneralPWMTimeoutCounter	
Description	The general PWM timeout counter value, timeout period = 65536 * the GPWM_TOC function clock period.	
Screenshot	General PWM Timeout Counter (0 -> 255) 	
Properties	Property	Value
	Type	INTEGER
	Label	General PWM Timeout Counter (0 -> 255)
	Default	1

3.2.4.23. GeneralPWMPolarity

Container	GeneralPWMPolarity	
Description	If enabled, the subinstance will invert the general PWM input.	
Screenshot	Enable invert the general PWM input 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable invert the general PWM input
	Default	false

3.2.4.24. MsgReadType

Container	MsgReadType	
Description	Define the how to read message. Interrupt or polling.	
Screenshot	Message Read type 	
Properties	Property	Value

	Type	STRING(RANGE)
	Label	Message Read type
	Default	Interrupt

3.2.4.25. SsiNotification

Container	SsiNotification	
Description	This parameter defines the existence and the name of a callout function that is called after a message is received.	
Screenshot	Ssi Notification 	
Properties	Property	Value
	Type	STRING
	Label	Ssi Notification
	Default	SsiNotification

3.2.4.26. QDTControllerId

Container	QDTControllerId	
Description	This parameter provides the controller ID which is unique in a given QDT Driver. The value for this parameter starts with 0 and continue without any gaps.	
Screenshot	Ssi Notification 	
Properties	Property	Value
	Type	IDENTIFIABLE
	Symbolic Name	true
	Default	None
	Origin	Flagchip